Automated Testing for React Web Application with Cypress

Bachelor thesis

Degree Programme in Computer Applications
Fall, 2022

Mohamed Al-Ajily

HAMK
HÄMEEN AMMATTIKORKEAKOULU
HÄME UNIVERSITY OF APPLIED SCIENCES

Computer Applications -koulutus Tiivistelmä
Tekijä Mohamed Al-Ajily Vuosi 2022
Työn nimi Automated Testing for React Web Application with Cypress
Ohjaaja Lasse Seppänen

TIIVISTELMÄ

Tämän opinnäytetyön tarkoituksena oli visuaalinen testaus ja testiautomaation kehittäminen Gemilo Oy:lle tapaustutkimuksena. Gemilon tuotekehitys etsi tapaa säästää tuotekehityksen ja ylläpidon kustannuksissa lisäämällä automaattiset visuaaliset testit MyGemilo-ohjelmaan. Tapaustutkimuksessa etsittiin tapoja toteuttaa automatisoituja visuaalisia testejä React-näkymiin, joihin on upotettu PDF-tiedostoja ja joissa on myös PDF:n esikatselu-näkymiä sekä ladattava PDF-tiedosto.

Opinnäytetyön tavoitteena oli jatkokehittää MyGemilon tuotekehityksessä jo käytössä olevaa testausprosessia. Pilvipalveluna ratkaisuja toimittava ohjelmistoyritys Gemilo Oy halusi parantaa lopputuotteen laatua ja vähentää tuotanto version mahdollisia teknisiä virheitä lisäämättä henkilöiden työaikaa testausresurssiksi käsin tehtävään ohjelmiston käyttöliittymän testaukseen.

Visuaalisista testeistä tuli osa pysyvä osa ohjelmistotuotteen laadun varmistusta. Tässä onnistuttiin. Visuaaliset testit varmistivat, että käyttöliittymä näyttää oikealta loppukäyttäjälle ja jokainen elementti verkkosovelluksessa on oikeassa muodossa, koossa ja sijainnissa. Työkaluksi visuaaliseen testaukseen valittiin Cypress-dokumentaatio. Tuotekehityksen kanssa oli helppoa toteuttaa visuaaliset testitapaukset sekä kehittää joitain testitapauksia MyGemilo-ohjelmaan. Opinnäytetyön aikana MyGemilo-ohjelmaan valmistui toimiva automatisoitu visuaalisen testauksen käyttöympäristö, joka vähentää tiimin työtä erityisesti PDF- ja sähköposti-alustojen testauksessa. Tuloksena on yksityiskohtainen kuvaus Cypressin automatisoidusta visuaalisesta testauksesta, ja työ auttaa testaajia ymmärtämään testiautomaatio-prosessia, peruskonseptin työkaluja ja Cypressin hyötyjä. Kirjoittaja tarjoaa myös vinkkejä, miten luoda luotettavampia skriptejä testaukseen

Avainsanat Cypress, visuaalinen testaus, automaattitestaus

Sivut XX sivua ja liitteitä XX sivua

HAMK
HÄMEEN AMMATTIKORKEAKOULU
HÄME UNIVERSITY OF APPLIED SCIENCES

Degree Programme in Computer Applications Abstract

Author Mohamed Al-Ajily Year 2022

Subject Automated testing for modern web applications

Supervisors Lasse Seppänen

ABSTRACT

This paper is about implementing visual testing and improving test automation at Gemilo oy as a case study. Gemilo's development team was looking for automated visual tests for HTML embedded templates such as PDF and email templates and to improve the existing testing processes in their running CRM application. These visual tests verify that the UI user interface appears correct to the end user and that each element of the web application appears in the correct shape, size, and position.

With the help of Cypress documentation and the development team, it was easy to implement the visual test cases and improve some test cases for the CRM application. The Gemilo development team was pleased with the final implementation of the test work. It appeared promising to them. Currently, the CRM application has a fully functional automated visual testing environment that reduces the team's effort in testing the PDF and email templates.

The work is mainly based on the author's personal experience as well as Internet sources such as Cypress' official website, e-books, and articles.

The result of the work is a step-by-step description of automated visual testing with Cypress. It can guide testers through the process of test automation. It can also help testers quickly understand the basic concept of the tool and the benefits of Cypress and test automation. It provides tips for creating more reliable test scripts.

Keywords Cypress, visual testing , Automated testing.

Pages	XX pages and appendices XX pages

Glossary

AUT	Application Under Test
CI	Continuous Integration
CRM	Customer relationship management
DOM	Document Object Model
E2E	End to End
ERP	Enterprise resource planning
GUI	Graphical User Interface
HTML	Hypertext Markup Language for web pages
QA	Quality assurance
ROI	Return Of Investment
SDLC	Software Development Life Cycle
SUT	System Under Test
TDD.	Test-Driven Development
UI	User Interface
URLs	Uniform Resource Locator
VSC	Visual Studio Code

Contents

1 Introduction 1
2 Research Objectives 3
3 Automated Testing Approach 4
3.1 Testing Techniques 4
3.1.1 Black-Box Testing 4
3.1.2 White-Box Testing 5
3.1.3 Grey-Box Testing 6
3.2 Testing Levels 6
3.2.1 Unit Testing 6
3.2.2 Integration Testing 7
3.2.3 System testing 7
3.2.4 Acceptance Testing 7
3.2.5 Regression Testing 8
3.3 Visual Testing 8
3.3.1 Manual Visual Testing 9
3.3.2 Automated Visual Testing 10
3.4 Automated UI Testing Tools 10
3.4.1 Selenium 11
3.4.2 Cypress 12
3.4.3 Comparing Cypress and selenium WebDriver 13
3.5 Cypress Testing Types 13
3.5.1 End-to-End 14
3.5.2 Component Testing 14
3.6 Test plan 14
4 Application Description 16
4.1 Why Cypress 16
4.2 Selected Testing tool 17
5 Cypress Test Structure 18
5.1 Folder structure 18
5.2 File Structure 19
6 Test plan and case implementation 22
6.1 Test plan at Gemilo 22
6.2 Gemilo’s Software Development Environment 24

6.2.1 Agile 24
6.2.2 Version Control 25
6.2 Test case improvement 26
6.3 Visual Testing implementation 27
7 Conclusion and Further development 29
7.1 Conclusion 29
7.2 Further development 30
8 References 31

Figures and tables

Figure 1 Spotting the difference in the Eiffel Tower image.
Figure Shows different web drivers control different browsers in Selenium.
Figure Shows how Cypress works in practice.
Table 1 Cypress Folder structure.
Table 2 Shows test case in practice.

program codes, command

code snippet 1 Common Structure of components tests.
code snippet 2 Changes made to the structure.
code snippet 3 Custom command for cypress/plugins/command.js.
code snippet 4 The improved custom command for cypress/support/command.js.

Code snippet 5 shows the imported plugin which is installed using “npm I cypress-image-diff-js/dist/plugin” to be used in visual testing.

code snippet 5 Visual testing plugin added to cypress/plugins/index.js.
code snippet 6 Sample code used for visual testing using the image diff plugin.

1 Introduction

Software testing is important because bugs and errors can be costly or even dangerous, potentially causing financial and human losses. History is full of examples, and one of them is the software bug incident in the United Kingdom. According to the Guardian newspaper, in April 2015, the Bloomberg terminal in London crashed due to a software bug, affecting more than 300,000 traders in the financial markets. This forced the government to postpone a £3 billion debt sale. Bloomberg stated that there were hardware and software failures in the network. (Nordeen, 2021, p. 12, Learn Software Testing in 24 Hours). Software testing is considered the most important phase in the software development life cycle (SDLC). Its main goal is to uncover undiscovered and hidden bugs in the software. Testing is defined as an activity to verify that the actual results match the expected results and to ensure that the software is free from bugs and defects. Software testing needs to be repeated during the software development process after every code change and every time the software is released/updated, which would cost software companies a lot of money and time. Therefore, the most important technique for software development testing nowadays is test automation, which is considered the most important technique for software development teams as it saves time and increases quality, efficiency, and test coverage. Moreover, it does not require developer intervention. Instead, developers can simply run the automated tests unattended.

Testing software is a constant challenge for web applications because they are constantly changing, their features are frequently added and removed, and they run on a variety of devices and browsers with varying levels of user performance and network connectivity. Test automation allows developers to test their applications as often as needed without having to worry about manually checking each change, and it helps them catch errors and bugs before they cause problems for end users. Automated testing can be used for different types of tests. Often, white-box testing is automated at the unit level and integration level. In test-driven development (TDD), developers are responsible for writing unit tests. These tests can be run when the code is checked into the source control system or in an automated build center on a schedule. Sometimes programmers are too busy to write unit tests or integration tests, and then a tester writes them. The advantage of test automation is that a test that has been automated once can be run.

automatically many times without using expensive testing resources. This can lead to potentially significant savings if the setup costs are less than the number of automated tests that are run. Another concept of test automation for web applications is visual testing. Whether you are a developer or a tester. There is always a need to ensure that the application under development maintains its visual appearance, and this concept is designed to give both developers and testers confidence that the user interface of the website has not been corrupted by the changes made by the developer. For example, in visual testing, there is no need to worry about a login button is missing from a version of the application deployed in the production environment, while it was present in earlier versions of the application.

These are the questions that the author aims to answer :

- What is Visual testing and how is it implemented?
- How to automate a test for an embedded HTML template like (PDF)?
- What is the impact of visual testing on a react web application?

2 Research Objectives

The goal of this thesis is to implement a visual test case for CRM applications and improve the existing testing process. In the work, customized visual testing / visual GUI testing was developed for the target company. The general idea is to track visual changes and ensure that unwanted visual differences are prevented before changes take place in production and identify how the user interface behaves in different configurations. The main goal is to implement a visual testing case for embedded HTML templates for a pdf and email form.

Planning for this work began shortly after I started my internship at Gemilo. When planning for visual testing began, there were a lot of factors to consider, but the experience and help of the development team made the planning phase smooth. With the current test automation, all software can be tested in the local environment. Visual testing is fully functional, and it tests pdf and prevents errors or visual changes to the pdf and email templates. The time required for testing has slightly decreased due to the improvement in test automation

What is left for future development is to execute test automation in CI/CD pipeline, called test-driven development (TDD), and integrate it with various CI. This way, every time the development team makes changes to the code base, the CI pipeline will automatically run all the tests to ensure that nothing has broken in production.

This thesis applies an action research study to answer the research questions. The research aims to introduce visual testing and improve the process of test automation, therefore action research is an appropriate methodology for this research. The author of the study is part of the Gemilo development team. During the project, many informal interviews/discussions took place with the development team and PO to discuss the framework and improvements to test automation at the process level. Limited resources were available for this case study at the beginning of the project, and the author is part of the project, so separate questionnaires are not required. Since the discussions were informal, I reflected on their views and primarily my own as part of the work. Through the discussion, the action research method was used effectively. These discussions/interviews helped to improve the test automation process in general and to select the best practices that fit into the current process.

3 Automated Testing Approach

A system that does not work properly is a huge drain on your patience, money, and time, and a bad experience for customers, and that's what makes software testing such an important part of the software development process. All it takes is one person making a mistake, and customers have a defect or bug in the software. The error could be in the code itself, in the instructions, or in the requirements that were originally communicated to the developers. (Why are tests necessary? ,2022).

Test automation software can input test data into the system under test, compare expected and actual results, and generate detailed test reports. However, in successive development cycles, the same test suite must be run repeatedly. With a test automation tool, you can record this test suite and replay it as needed. Once the test suite is automated, no human intervention is required. This improves the ROI of test automation. The goal of automation is to reduce the number of test cases that need to be executed manually, not to eliminate manual testing altogether.

When asked by the author to the development team what they hope to gain from test automation, the most common response is that it can reduce the time it takes to test the software before it is released, and that testing helps identify bugs and defects before they are delivered to the customer, reduces errors in the system, increases the reliability and quality of the system, and makes it user-friendly.

3.1 Testing Techniques

There are different techniques that can be used for software testing such as black box testing and white box testing and grey box testing.

3.1.1 Black-Box Testing

Black box testing is a technique of testing without having any knowledge of the interior workings of the application and the tester does not know the system architecture and does not have access to the source code. Typically, when performing a black-box test, a tester interacts with the

system's user interface by providing input and examining the output without knowing how and where the input is processed. (Hamilton, 2022)

The goal of black-box testing is to verify the correctness of software behavior that directly supports daily business activities. This goal is often referred to as behavioral coverage. The requirements for black-box testing are the software requirements, use cases, and only the executable program and its data. These requirements are usually met in the middle stages of the development life cycle when large pieces of code collaborate or when software is purchased from a vendor. It makes sense for anyone (developer or tester) to perform black box testing if the person performing the testing is not the person who wrote the code. Software developers typically focus their black box testing on checking the expected behavior of the code (positive testing) against use cases, which tends to cause them to ignore unexpected behavior (negative testing). Testers increase the value of black-box testing by planning and executing both positive and negative behavioral tests, knowing that most user-detected bugs result from unexpected behavior. (Everett & Jr, 2007, p. 112).

3.1.2 White-Box Testing

White box testing is a testing technique that tests the internal structure, design, and coding of software to verify input-output flow and improve design, usability, and security. In white box testing, the code is visible to the testers, hence it is also called clear box testing, open box testing, transparent box testing, code-based testing, and glass box testing. (Hamilton, 2022). The goal of white-box testing is to verify the correctness of the software's instructions, code paths, conditions, loops, and data flows. This goal is often referred to as logic coverage. Prerequisites for white-box testing include software requirements, use cases, the executable program, its data, and its source code.

The software developer typically performs white-box testing as an extension of code debugging activities early in the development cycle. Software developers typically focus on getting the code to work according to the use cases, which leads them to debug only the code they know works (selective logic test coverage). Testers increase the value of the developer's debugging activities by helping them plan and debug a larger portion of the code than usual (more thorough logic test

coverage). The more logic test coverage you achieve during debugging, the fewer bugs will be discovered later by other types of testing. (Everett & Jr, 2007, p. 107).

3.1.3 Grey-Box Testing

Grey box testing is a software testing technique for testing a software product or application with partial knowledge of the internal structure of the application. The purpose of grey-box testing is to search for and identify errors due to improper code structure or improper use of applications. This process usually identifies context-specific errors that relate to web systems. It increases test coverage by focusing on all layers of a complex system. (Hamilton, 2022).

In software engineering, grey box testing offers the possibility to test both sides of an application, the presentation layer which is the user interface of the application as well as the code part. It is especially useful in integration testing and penetration testing. This kind of technique is considered a combination of White Box Testing and Black Box Testing.

3.2 Testing Levels

There are different types of stages in the process of software testing. In this chapter, a brief description of the testing levels is given. The testing levels include various methods that can be used when performing software testing.

3.2.1 Unit Testing

In unit testing, code is broken into logical parts to test it, and generally, only one method is tested at a time. A unit can be considered the smallest possible piece of code, such as a function or method. This means that ideally, each unit should be independent of all other units. When we run unit tests, we try to test each function or method as soon as we complete it to make sure that the code we worked on works before moving on to the next function or method.

In unit testing, each module is tested individually. It follows white box testing and helps fix bugs early in the development cycle and save costs. It helps developers understand the test code base and allows them to make changes quickly. (*Levels of Testing*, n.d.).

3.2.2 Integration Testing

Integrated testing refers to the testing of all the different components of our program. This is to ensure that functional, performance, and reliability requirements are met. We also test the different units together and see if they can work. Integration testing can take different forms, such as top-down and bottom-up approaches.

The integration test case differs from other test cases in that it focuses mainly on the interfaces and data/information flow between modules. Here, the integrating connections take precedence over the unit functions that have already been tested. Integration tests focus on the integration of two or more modules, i.e. on the communication between modules, and they follow a white-box test. (*Levels of Testing*, n.d.).

3.2.3 System testing

System testing is the third stage of software testing. The main goal of this stage is to test the software functionality and non-functionality requirements, it is known as end-to-end testing, which means that the test environment is parallel to the production environment. System testing means that the application is tested as a whole system.

System tests are performed on a complete integrated system to assess the conformance of the system to the specified requirements, i.e., confirming that the system as a whole delivers the functionality originally required, and they follow black-box testing. . (Levels of Testing, n.d.).

3.2.4 Acceptance Testing

The acceptance test is usually the last phase of the entire testing process. It is often performed before the final acceptance of the software by the customer. It can be divided into two parts. First, the software vendor performs the acceptance testing, then the end-users. Acceptance testing is when your customer (end-user) actually tests the software you have created. A typical process involves the end-users creating test cases that reflect the business use of the software.

The main purpose of UAT is to validate the entire business process. It does not focus on cosmetic errors, spelling errors, or system testing. User acceptance testing is performed in a dedicated test environment with production-like data structure. This is followed by black-box testing involving two or more end users such as the customer and the end users. (*Levels of Testing*, n.d.)

3.2.5 Regression Testing

Regression testing is one of the black box testing techniques. The main idea behind it is that a code change in the software does not affect the existing functionality of the product. It ensures that the product works correctly with the new bug fix or features implemented by the development team. A regression test is also called a verification method because it verifies that the newly implemented code does not affect production. Regression testing means re-executing test cases that were matched in the past with the new version to ensure that the app's features work correctly. Also, regression testing is a series of tests rather than a single test that is run every time you add new code. (*Levels of Testing*, n.d.).

3.3 Visual Testing

Nowadays, visual testing plays an important role in software testing. Visual testing is a way to compare the actual image displayed on the screen as the user sees it with the base image already stored. (Axelrod p. 436)

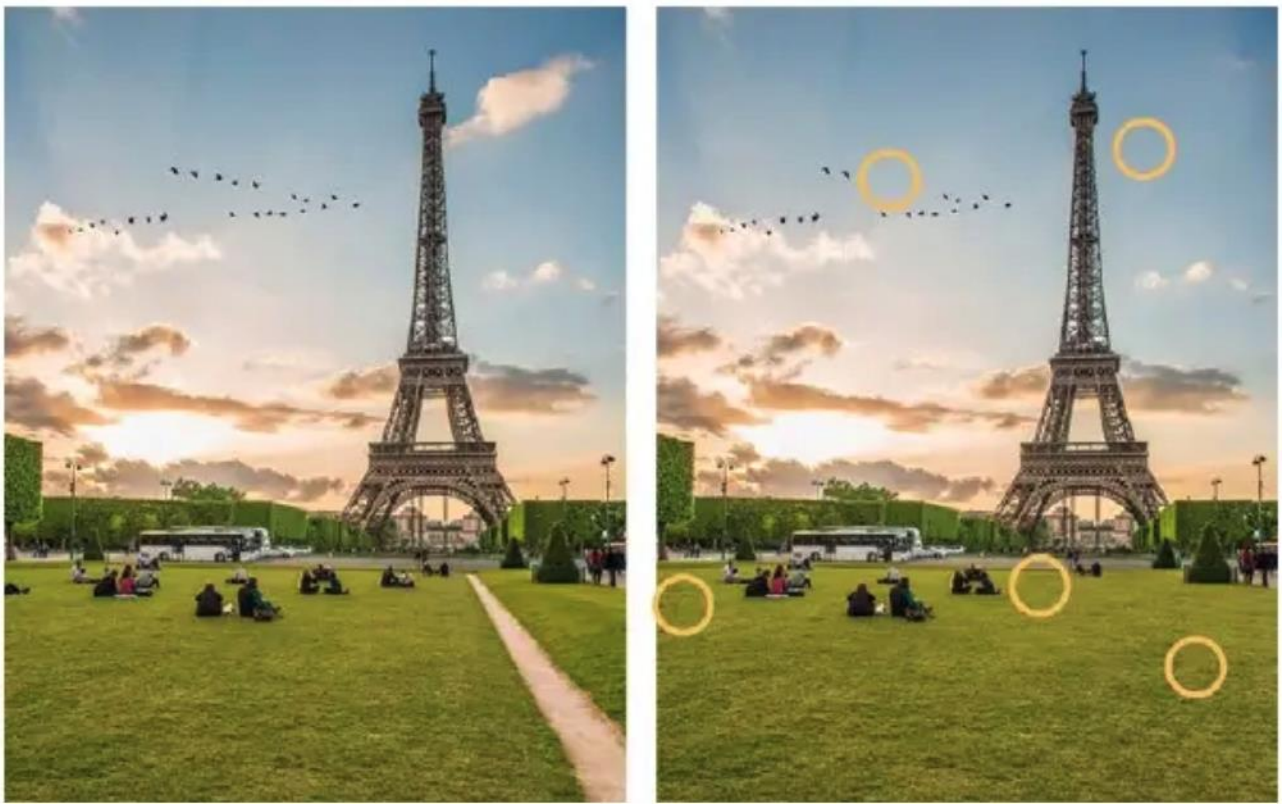
One might wonder if visual testing is about comparing two images, and if our eyes are accurate enough for validation, why do we need it? Well, the naked eye can perceive visible page changes, but it is harder for the eye to detect minute details, such as CSS property changes that have moved an input element several pixels, or minimal pixel changes.

Visual testing served to reassure developers and testers that the UI website was not corrupted by the changes made by developers. To give you an idea: With visual testing, you do not have to worry that the delivered product is missing a color or a button while it was present in the previous production. In the next two subchapters, we will go through the types of visual testing. (Mwaura p. 194).

3.3.1 Manual Visual Testing

Manual testing involves checking with the naked eye that changes made in production do not affect the functionality of the user interface. This test is usually performed by testers and developers by comparing the developed products with the originally created products. Manual testing may be good and suitable for small UI changes, but it is not very accurate for validating an application with many pages and large changes or different viewports in the user interface. Take a few seconds to compare the following images published by Atlantide Photo Travel showing a side-by-side comparison of the Eiffel Tower and try to find visual differences.

Figure 1 Spotting the difference in the Eiffel Tower image



Looking at the details of the two images, there are such bird patterns, missing people, and even missing clouds that could make it visually impossible to tell a difference between the two images. If manual tests were applied to large applications, details would most likely be missed and it would be difficult to find differences when comparing. . (Mwaura,2018 , Manual Visual Testing p. 43).

3.3.2 Automated Visual Testing

Automated visual testing involves testing the visual elements of the UI website. Instead of the manual visual testing approach, an automated process is used to test the stability of the user interface. To apply the visual testing approach, we need to define our initial image(Baseline image) before testing. Then, when we run the test, a screenshot of the current website UI is taken and compared to the initial image(baseline image) we have. The test tells us if there are any differences between the two images by showing the percentage threshold that indicates how different they are. (Mwaura,2018 , Automated Visual testing p. 44).

3.4 Automated UI Testing Tools

Automated testing is the use of specialized software, separate from the software under test, to control and execute tests, including comparing and reporting actual results with predicted results. The application is referred to as an "Application under Test" (AUT) or "System Under Test" (SUT), and the software used for testing is referred to as an "Automated Testing Tool" (ATT) (Boby, 2021, Test Automation: A manager's guide, p. 41).

Automated tests are there to automate your software. These tests interact with your application and compare the actual output to the expected output. They are successful when the output is correct and give you meaningful feedback when it is not.

Automated tests reduce the gap between writing code and receiving feedback. Therefore, they make the development process more structured and reduce the number of defects. A predictable development process makes it easier to estimate tasks and allows developers to revise their work less often. When tests are automated, rework and communication efforts reduce by themselves, and developers can immediately review the work of others and ensure that they have not broken any parts of the application. (da Costa, 2021, p. 61).

The success of automation testing depends on the selection of the right test automation tool and is one of the most important factors in achieving the project goal. There are many types of tools that software companies can consider, such as open-source tools, commercial tools, or custom tools. There are quite a few test automation frameworks such as Selenium, Cypress, Appium,

Cucumber, Citrus, etc. In the next chapters, we will discuss the most popular and open-source frameworks like Selenium and Cypress.

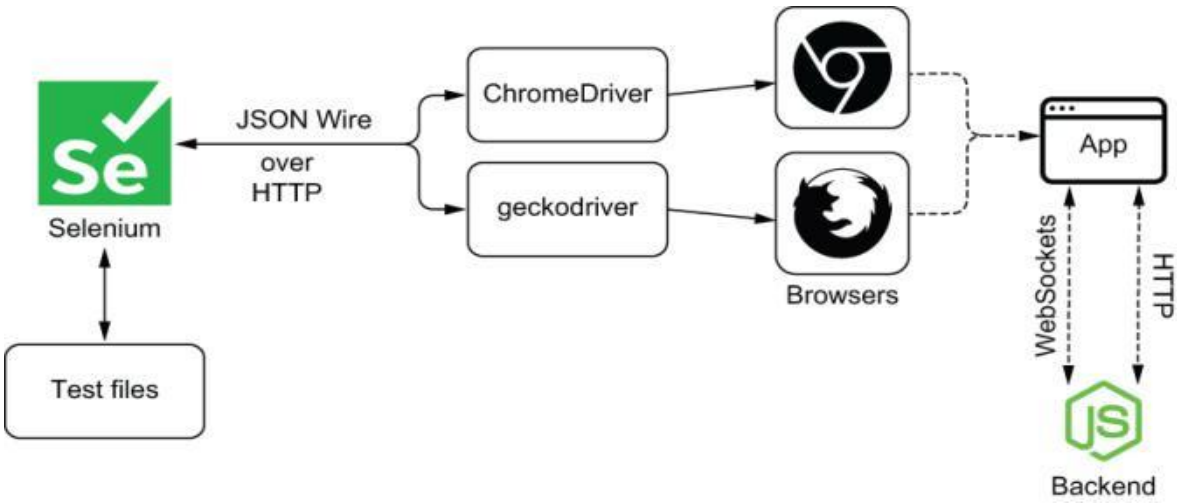
3.4.1 Selenium

In 2004, tester Jason Higging was testing an internal application at ThoughtWorks (the company that specializes in agile software development) when he realized he could better allocate his time to manual testing activities. To that end, he developed a JavaScript library that interacted with the browser and would soon interact with other browsers. This project was named Selenium, an analogy to an alternative testing tool (open source) for testers who could no longer rely on Mercury's paid and expensive Quick testing tool. In chemistry, Mercury's antidote is called Selenium. That's where the name comes from.

In 2006, a Google engineer, Simon Stewart, decided to explore the Selenium library further and founded the WebDriver project, which was eventually merged with the old library (Selenium RC), resulting in the second version of Selenium in 2008. Today, Selenium is a suite of tools: Selenium IDE, Selenium Remote Control, Selenium Grid, and Selenium WebDriver. Each has its own purpose, but a common goal of automating functional testing in a practical and efficient way. (Peres,2018, p.50).

Nowadays, Selenium is the most popular test automation tool and is especially suitable for UI web application automation. It allows mimicking mouse and keyboard actions on behalf of the user and retrieving data that is displayed. Selenium has some important advantages that make it so popular. Some of them are that it is a free open-source code and supports a large number of browsers. It is also available in many programming languages. (Axelrod, 2018, Complete Guide to Test Automation, p. 51). The figure below shows that different web drivers control different browsers in Selenium.

Figure 2 Shows that different web drivers control different browsers in Selenium.

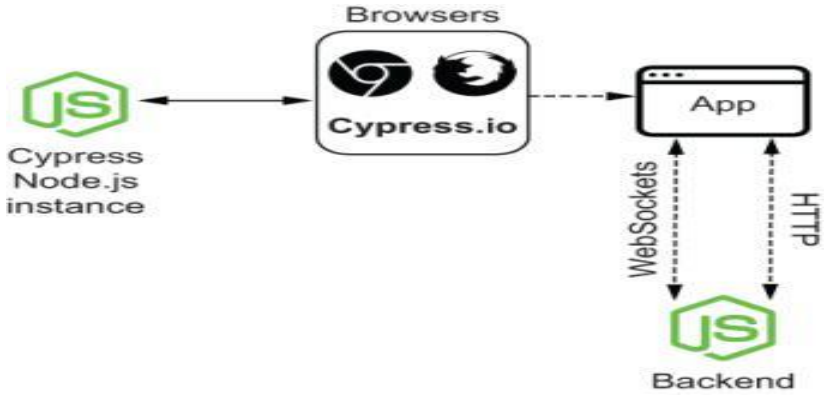


3.4.2 Cypress

Cypress is a free and open-source automation tool for software testing. It is written in JavaScript and is used by well-known and international organizations such as NASA and DHL. It is an end-to-end testing framework written by developers and used for testing web applications (why Cypress, 2022).

Cypress focuses on eliminating inconsistencies in tests by ensuring that you can write, debug, and run tests in the browser without requiring additional configuration or packages. Cypress works as a standalone application and can be installed on macOS, Unix/Linux, and Windows operating systems using either hyphenated applications or command-line tools. Cypress is designed primarily for developers who write their code in JavaScript, as it can be used to test any application that runs in a browser. The following figure shows how Cypress works in practice.

Figure 3 Shows how Cypress works in practice



3.4.3 Comparing Cypress and selenium WebDriver

Understanding why Cypress and Selenium web drivers are different will help us understand different and similar aspects of the two frameworks. Cypress and Selenium are often compared, but they are both different and not subject to the same constraints (Cypress, 2022). Cypress was developed for the web and is optimized to run in browsers. Not all major browsers are currently supported, as is the case with Selenium Web Driver. Cypress currently supports browsers developed with the Chromium open-source project, as well as Firefox, Chrome, Edge, and Electron (default browsers in Cypress), while Selenium supports all major browsers, which is an advantage when it comes to the ability to test applications on multiple platforms. Selenium offers users the ability to extend it with different browsers and plugins and supports different languages such as Java, Python, Ruby, C#, JavaScript, and even mobile automation in tools like Appium. Cypress focuses solely on improving testing for JavaScript web developers and QA engineers who understand the JavaScript language. (Mwaura p.79) Architecturally, Cypress runs in the browser, which means it is significantly faster and can interpret commands at runtime more quickly because there are no third-party services interpreting commands on its behalf or sending an HTTP request to the browser driver.

3.5 Cypress Testing Types

There are two types of testing that cypress offers, end-to-end testing, and component testing. Deciding which type to use when creating the test depends on the benefits and considerations of each type of test.(Testing Types, n.d.)

3.5.1 End-to-End

This type of testing is referred to as E2E. It is a technique that tests our application from the front-end, which is the web browser, to the back-end, which is the application server. One of the advantages of E2E testing in Cypress is that the test is run in the same way that users interact with the application, by using the Chromium browser, visiting URLs, filling out forms, clicking buttons and links, selecting a date, etc. This ensures that the test and the user experience are the same. E2E tests can be written by QA teams or developers and used for integration testing. (What is E2E Testing, n.d.)

3.5.2 Component Testing

When writing applications in modern web frameworks, they can be broken down into small units called components, which can differ in size, such as buttons, which are quite small, and forms, which are larger components. Due to their nature, they are usually easy to test. This is where component testing plays an important role in testing.

Component testing is different from E2E. They can be assembled and tested independently, while E2E visits URLs to test an entire application, and this allows us to focus on a single component function at a time. It can be written by developers working on that component to ensure that the functionality of the component is tested as it is built with component testing, it is important to remember that even if all component tests pass, it does not mean that the system as a whole is working properly, and it says nothing about whether all layers of the application are working well together. Therefore, a well-tested application consists of a combination of E2E and unit tests. . (What is Component Testing, n.d).

3.6 Test plan

In recent years, the view of software testing has matured into a constructive one. Testing is no longer seen as an activity that starts only after the coding phase is complete and only aims to identify defects. Software testing is or should be frequent throughout the development and

maintenance life cycle and test planning should begin early in the software development process, test plans and procedures should be systematically and continuously developed and, where possible, improved as software development progresses. These testing and test planning features provide useful information to software designers and help highlight potential weaknesses such as design violations/inconsistencies or documentation gaps/ambiguities. (Naik and Tripathy, 2008, p. 21).

Like all other aspects of project management, testing activities must be planned. The key aspects of test planning include coordination of personnel, availability of test facilities and equipment, creation and maintenance of all test-related documentation, and planning for possible undesirable outcomes. If more than one baseline of the software is being maintained, then a major planning consideration is the time and effort needed to ensure that the test environment is set to the proper configuration.

A test plan is a detailed document that describes the test strategy, objectives, schedule, estimation, deliverables, and resources required to perform testing for a software product. The test plan helps testers determine the effort required to verify the quality of the application under test. Test Plan. Serves as a blueprint for conducting software testing as a defined process that is closely monitored and controlled by the test manager

The importance of the test plan is that it promotes understanding of the details of testing outside the test team, such as developers, business managers, and customers. And it is a set of rules that testers must adhere to. The test plan documents important aspects such as test estimation, test scope, and test strategy so that they can be reviewed by the management team and reused for other projects. (Hamilton, 2022).

Test planning and defect tracking is essential for software testing. Good management of test planning and tracking can shorten test cycles by facilitating collaboration and communication among team members, increasing test effectiveness, and making the testing process more visible by allowing team members to track issues and assess application readiness.

.

4 Application Description

The target company is a software company allocated in Tampere and was established in 2007. It delivers hosted SaaS service and customized CRM and ERP solutions to the business-to-business market.

The target company has a CRM application that is implemented, improved, managed, and maintained based on customer needs. The application is a solution that helps monitor who is selling what, to whom, and at what price. The target company has developed CRM as a sales management system that supports B2B sales. It is suitable for large deals that may take a long time to close due to the preparatory work required.

The target company’s application has various features such as managing daily, weekly and monthly plans for deals and salespeople, as well as pipelines/Kanban. The application has a time-tracking feature that makes it easy for a company's employees to record their working hours and calculate their monthly working hours. There is also invoice tracking for managing business invoices.

The target company application offers sales statistics that help predict sales. It shows the total volume of sales, salespeople, activities, and results by the salesperson. This makes it easy for a company's management to assess why some salespeople are successful and others are slower at closing deals.

4.1 Why Cypress

The target company does not have its own testing team, and Cypress was easy to install and use. In addition, React is a JavaScript-based programming language, and Cypress was designed for JavaScript developers. It is also simple and open source, and at the time of installation was one of the most popular testing frameworks in modern software development.

Moreover, once cypress is installed and initialized, everything is ready to go without any further configuration, and the team can start writing automated tests to ensure that the most important

features of the application are covered. It even brings executable examples of how to use each feature to help developers test a wide variety of situations found in modern Web applications. In an interview with the company's architect, the architect said Cypress offers an excellent development experience, such as watch mode, which means tests are re-run after a test file is saved, and time travel, which helps developers debug tests by saving snapshots of the application before and after each action and allowing them to go back in time to see exactly what happened while the test was running. Cypress also takes screenshots and videos when a test fails and provides an automatic wait feature by default, meaning it waits for elements to be visible before attempting to interact with them, and it waits until requests are finished before proceeding with tests. It even waits for animations to complete. The Cypress framework captures snapshots at the time of test execution, allowing developers to hover over a particular command in the command log to see exactly what happened at that step.

4.2 Selected Testing tool

The target company is developing CRM applications using Reactjs as a front-end and FastAPI as the backend programming language and prior to this research, Cypress has already been in use by Gemilo developers.

5 Cypress Test Structure

Creating and writing test suites for software testing is called test design. Test architecture and test condition identification provide us with a general idea for testing that applies to a variety of scenarios. However, when it comes to creating a test case, it should be more specific. In this project, the test files were created based on the individual functions of the application. Since there are multiple functions, the test files were divided into subfolders. In this way, the files are clear and easy to view. Below is an explanation of the folder and file structures.

5.1 Folder structure

To write tests in Cypress, it is important to understand its folder structure. After installing Cypress in your React application by running " npm install -save dev cypress"," Cypress.json", the Cypress folder is created by default and appears in the file hierarchy of the code editor, in Gemilo's case we are using Visual Studio Code (VSC) from Microsoft. The Cypress folder contains several subfolders like:

Table 1 Cypress Folder structure

Folder name	**Description**
Fixtures folder	used to store test data and is usually written in JSON format
Integration folder	contains all test files (. spec.js). All test files should have the extension (. spec.js)
Support	folder contains the "index.js" and "commands.js" files, which are used to place custom commands or global commands that we do not want to repeat for each test
Plugins	used to extend the behavior of Cypress, for example by adding plugins tailored to our needs. And it contains an "index.js" file that is automatically imported for each test
Screenshots	cypress can automatically take screenshots when a test fails, when you run "cypress run" or even when you manually take screenshots with "cy.screenshot()". These screenshots are stored in this folder

5.2 File Structure

The test files were created based on the components. Each component has different functionality; however, all tests share a common structure. The code snippets will show the common skeleton of the tests before any enhancement takes place.

code snippet 1 Common Structure of components tests

```
describe('add an installment to a deal', () => {
before(() => {
cy.loginByAuth(
Cypress.env("auth_username"),
Cypress.env("auth_password"))
})
beforeEach(() => {
Cypress.Cookies.preserveOnce("token")
})
```

Code snippet 2 shows the first improvement case made to the structure of the tests which contains describe() function that wraps the whole test and the beforeEach() which wraps the function that logs in into the system.

code snippet 2 Changes made to the structure

```
1 describe("add an installment to a deal", () => {
2     beforeEach(() => {
3         cy.login("sales");      You, 3 days ago • mopdifed abd improved tests …
4     });
```

Code snippet 3 shows the custom command used for logging into the system and how sessions are being handled by cypress and functions that is used for mimicking the users actions such as visiting a website URL.

code snippet 3 custom command for cypress/plugins/command.js

```
Cypress.Commands.add("loginByAuth", (username, password) => {
cy.clearCookie("token");
cy.visit("/login");
```

```
cy.get("input[name=username]")
.type(username)
.should("have.value", username);
cy.get("input[name=password").type(password).should("have.value", password);
cy.get("form").submit();
cy.url().should("include", "/intra");
});
```

Code snippet 4 shows the changes made by the author to the custom commands by adding multiple users that have different permissions and privileges, and how session is handled to be saved before each test run.

code snippet 4 The improved custom command for cypress/support/command.js

```
Cypress.Commands.add("login", (user) => {
const USERS = {
admin: {
username: "admin@test.test",
password: "admin",
},
sales: {
username: "matti@test.test",
password: "matti",
},
};
cy.session(user, () => {
let username = USERS[user].username;
let password = USERS[user].password;
cy.visit("/login");
cy.get("input[name=username]").type(username);
cy.get("input[name=password")
.type(password)
.should("have.value", password);
cy.get("form").submit();
cy.url().should("include", "/intra");
});
});
```

Code snippet 5 shows the imported plugin which is installed using “npm I cypress-image-diff-js/dist/plugin” to be used in visual testing.

code snippet 5 visual testing plugin added to cypress/plugins/index.js

```
module.exports = (on, config) => {
config.env.auth_username = "admin@test.test";
config.env.auth_password = "admin";
require("@cypress/code-coverage/task")(on, config);

// `on` is used to hook into various events Cypress emits
// `config` is the resolved Cypress config
const getCompareSnapshotsPlugin = require("cypress-image-diff-js/dist/plugin");
getCompareSnapshotsPlugin(on, config);
return config;
};
```

code snippet 6 sample code used for visual testing using the image diff plugin.

```
const retryOptions = {
 limit: 5, // max number of retries
 delay: 500, // delay before next iteration, ms
 };



 cy.get(
 "[data-cy=reorderable-item-cover_page-content]"
 ).compareSnapshot(
 `pdf_with_kansilahti${Cypress.browser.name}`,
 0.02,
 retryOptions
 );
```

In the next chapter, the code snippets will be described in detail.

6 Test plan and case implementation

In this chapter, the author of the thesis intends to present test planning at Gemilo, and the implementation of the case study and the working environment of the target company. Practices, i.e., daily work and following daily tasks, were carefully considered when applying new test cases or improving existing test cases. The overall implementation of the work case was done in steps with defined objectives so that the goals could be easily achieved.

6.1 Test plan at Gemilo

In Gemilo, the product owner is responsible for the test plan and backlog. PO creates the product roadmap and prioritizes the delivery team's deliverables to the business. PO also defines the product features, called user stories in Scrum, which are added as tasks to the backlog. After analyzing the user stories, PO creates a ticket and trancripts (notes) test cases in the agile framework. In most cases, gemilo developers test their own code as they work on it. This practice is called unit testing and usually involves the tester themselves running individual tests for each function and class. However, every time a developer adds a new function to the application, it could affect a previous project function feature. For this reason, testing is often performed during the development process and again as each component of the application is completed and with each update release

Each time a task is developed by the developer, the tester receives a copy to check for possible bugs. The tester starts a new task and returns it to the developer if there are problems with any part of the application. After a development process is complete, the tester can start working and provide all the findings insights and test results before the start of the next sprint as soon as the developers report their tasks as completed. If the development phase of the next sprint begins before testing of the previous sprint is complete, developers may need to fix issues from the previous sprint while working on the new sprint. The automated tests are started by the tester after the development phase is complete. After the tester starts running the tests and submits a report confirming that the tests have been run and passed, it is time to hand over the project over to the customer.

When it comes to managing the tests, a test case can easily become too large and full of outdated test cases if not managed regularly. Multiple test cases are run by the tester, reported as failed, and then closed by PO, only to waste unnecessary time and resources. A test case is written based on the user story, and a few sprints later a new user story changes the workflow or intended functionality, and a new test case is written. If the previous test is not updated or deleted, it can lead to a number of confusions between testers, developers and the product. To avoid this situation, it is not necessary to create new tests when the workflow is updated but, for example, no significant changes are made to the functionality. It is sufficient to edit the tests so that they match the functionality in the test suite and take them into the sprint and adapt them to the new workflow.

If only one image, color scheme or text is adjusted in the visual tests, such as the embedded HTML templates, the base images should be updated based on the new user story. Otherwise, the visual tests will fail because they depend on the threshold percentage and the base image, and the comparable image should be the same.

The creation of test cases follows the same pattern. Each test case must have a ID, in our case the same test task id assigned to the tester, as well as a descriptive title, precondition, test steps and expected results. The following table shows an example of a test case used in gemilo.

Table 2 Shows test case in practice.

ID	12325
Pre-condition	Visit URL
Title	Login Test
Test Steps	Enter correct values
Test Steps	Enter Username Enter Password Click Login
Expected Results	Page informs about login sucess
Test Steps	Enter incorrect values
	Enter Username

Test Steps	Enter Password Click Login
Expected Results	Page informs about login failure

Finally, Gemilo releases a version update every week. With each release, the development team runs all the test cases for the update version to ensure that the bug fixes and new features do not affect the functionality of the application.

6.2 Gemilo's Software Development Environment

A local development environment has been already set for writing the tests. This was implemented by the development team on a Linux-based operating system, React is the front-end framework for programming language. And Fast-API as the framework for backend development. All the environment was installed locally on the company computer after following the installation instructions of the commissioning company and then the implementation phase took place.

6.2.1 Agile

Agile is a set of methods and methodologies that help your team think more effectively, work more efficiently, and make better decisions. These methods and methodologies address all areas of traditional software development, including project management, software design and architecture, and process improvement. Each of these methods and methodologies consists of practices that have been streamlined and optimized to make their adoption as easy as possible. Agile has dozens of methodologies to choose from such as Scrum, extreme programming (XP), Lean, and Kanban and they all aim to change the development team's mindset from individuals who simply follow a plan to a cohesive group that makes decisions together. (Stellman & Greene, 2014, p. 2).

Agile is also a mindset because the right mindset can have a big impact on how effectively a team uses the practices. This mindset helps people on the team share information with each other so

they can make important project decisions together instead of having one manager make all those decisions alone. An agile mindset is about opening up planning, design, and process improvement to the entire team. An agile team applies practices so that everyone shares the same information and everyone on the team has a say in how the practices are applied. In Gemilo, The Kanban method is used and there is a dedicated framework where the development team performs its daily development, testing, and bug-fixing tasks. The development team implemented the framework on day one. When the author receives a task, the company's notification system notifies them that there is a new task. After review by the architect or another developer. The task status changes to "completed".

6.2.2 Version Control

Gemilo uses Git version control, which is free and open source, designed for small and large projects. In Gemilo, there is a master branch called "next-update". In this branch all branches are merged. Each developer creates their own remote branches using "git checkout -b" from the master branch after fetching the latest master branch with "git fetch" and pulling it with "git pull". This avoids code conflicts and ensures that the origin/master branch is up to date.
Gemilo has its own naming style for branches, where the name of the branch is the name of the task assigned to the branch, with a descriptive title next to the developer, e.g., "33334-dropdown-styling-task". After the task is complete, the developer adds the modified code to Git using "git add " to add a specific file or "git add ." to add all modified files.

After adding the files, the developer can check the version control status by using "git status" which shows the name of the files that have been modified, changed, created or even deleted. Then the code is committed with a descriptive message that informs about the changes that the commit makes to the behavior of the code, rather than what has changed in the code, using "git commit -m "the commit message". then after comparing and viewing the changes the developer can finally push the changes to Use "git push ".

6.2 Test case improvement

In the Gemilo project, each test has the same structure, which makes it easier to read and write test cases. In **code snippet** 1 Common Structure of components tests, the structure is divided into three parts. The first part describe() is a function of Mocha provided by Cypress and is basically used to group our test. We can nest our test as deeply as we need. This function takes two arguments, as shown in the figure. The first argument is a string representing the name of the test, and the second argument is a callback function. Inside the describe() function, there is a before() function that calls a function configured to log into the site with a correct username and password, in the command.js file located in Cypress/Support/command.js. This function opens a URL page for the login form before we run all our tests. After that, we call the beforeEach() function. This function is a root-level hook and is executed inside the describe() function before each test; it allows you to intervene in the module to change its action before anything happens. In our case, we call "Cypress.Cookies.preserveOnce("token")" to preserve the cookies, since by default Cypress deletes all cookies before each test to prevent the state from building up.

As mentioned in the file structure in Chapter 4, the support folder contains the command.js file, where we store our own commands. Cypress provides the syntax (Cypress.Commands.add (name, callback function)) by default, which is used for adding custom commands.

The first line of code is (Cypress.Commands.add), which is provided by Cypress to define or override commands that can be used in our tests. This is followed by the string "loginByAuth", which in this case is the name of our function that takes two arguments (username, password). The next line is cy.clearCookies("token"), which clears the cookies, and then cy.visit(), which visits a remote URL, which in our case is the URL of the login form to log in to the project.
To populate the login form, we use cy.get(), a command used to retrieve DOM elements in Cypress. In our case, we get the username field of the element and use cy.type() to enter the username in it.

In code snippet 2 Changes made to the structure, instead of using the before() function, we use the beforeEach() function, which contains a function that logs in before each test run. We got rid

of this function because it logs in and out before each test run, which means it takes longer to run the test.

In code snippet 4 The improved custom command for cypress/support/command.js**,** we changed the logic a bit and added the user's object, which contains different users to be used in the login form. We also added cy.session(), which is used to cache and restore cookies and session memory to reduce the setup time of the test. This means that we no longer need to keep cookies and then log in and out during the test, as cy. session() saves the session and restores it as needed during the test.

6.3 Visual Testing implementation

Gemilo did not have any kind of visual testing for its HTML templates and other templates. Therefore, the author implemented the visual testing using a tool provided by cypress called "Image-diff-js". After installing the plugin from cypress, run the command " npm i cypress-image-diff-js". The plugin creates a folder in cypress called "cypress-visual-screenshots" inside the cypress folder, which contains three folders for storing screenshots. The first folder is called "Baseline", where the images captured by the plugin are stored, the second folder is called "Comparison", where the baseline image is compared with the recently captured image, and the last folder is called "Diff", where the visual differences in pixels between the baseline and comparison images are displayed. The idea of the plugin is that after calling the command we saved in command.js in our test file (spec.js), it first checks if the target element already has a baseline image. If it has a base image, it takes another screenshot for comparison, and if not, it takes a screenshot for the target element and saves it in the baseline folder, and then takes another screenshot and compares it to the base image for the target element. In

Code snippet 5 shows the imported plugin which is installed using “npm I cypress-image-diff-js/dist/plugin” to be used in visual testing.

code snippet 5 visual testing plugin added to cypress/plugins/index.js, the code only indicates that we have imported and initialized the cypress image command.

In code snippet 6, the code structure is divided into two parts, let us go through the first part. The first line is "const retryOptions", a JavaScript function with two parameters. The first parameter is "limit:5", which refers to the maximum number of attempts for the plugin, in our case, it is 5, which means that the plugin will try five times if it fails. And the second parameter is "delay:500 " which specifies the delay before the next iteration and in this case, it is 500 milliseconds.
The second part of the code snippet is how to use the plugin to take the screenshot. The first line is cy.get(), which in this case asks Cypress to select the element from which to take the screenshot, and then compareScreenShot(), the plugin function that takes two parameters: the name we want to set for the screenshot, and the name of the browser, since we are testing with different browsers. For this reason we used (Cypress.browser.name), which stores the name of the browser next to the name of the screenshot.

7 Conclusion and Further development

This chapter contains a brief description of the results of the work and its future development. The final section explains the work carried out for the target company and the current state of development. The further development elaborates the future plans and the improvement of the test environment and provides a basic step for the implementation.

7.1 Conclusion

The importance of software testing as part of the software development process cannot be overstated. The goal of software testing is to find software defects and ensure that the software meets the business and technical requirements placed on it. To achieve this goal, it is important to perform software testing comprehensively at every stage of development. In this work, the background of test automation and software testing, in general, was explained and end-to-end testing cases for a modern web application were implemented using the Cypress tool and end-to-end testing was added as part of continuous integration. The goal of the paper was to provide a functional and comprehensive overview of the above topics. Cypress is the latest testing tool and is great for testing modern web applications. Also, the visual testing plugins offered by it are well-suited for end-to-end testing. The work gave a comprehensive overview of how visual testing is implemented for a modern web application using the Cypress tool, and end-to-end testing was added as part of continuous integration.

The primary goal of the thesis has been achieved by utilizing the cypress plugin to write and run automated testing for embedded templates, the user interface of the CRM applications can be tested in a much shorter period of time. In addition, many custom commands have been written and can be used to serve specific testing purposes. The result was a report that can be used to help implement end-to-end testing for a modern web application.

7.2 Further development

Gemilo's current state of test automation runs in UI mode using the Cypress Dashboard. Tests are run manually before each update to ensure that the development team's work has not affected the functionality of the application. The visual tests work as expected and prevent the occurrence of bugs and errors when the test is run locally. However, it fails when run on a different computer because the percentage of thresholds and screen resolutions are different. The further development of test automation is to run the tests in the CI/CD pipeline. This way, whenever a change is made to the code base, the CI/CD pipeline automatically runs all the Cypress tests to ensure that nothing has broken in the application.

Cypress provides extensive documentation on using Cypress in CI and integrating with various CI providers. Currently, Gemilo production runs in a Docker container and the idea is to run the tests in the docker container using the official Cypress Docker image and then integrate the tests into the Jenkins CI environment, which is an open-source CI environment where tests can be automated as part of the build process. Running the tests in CI saves the company time to run the tests and prevents buggy code from entering production, ensuring that the customer does not experience user interface errors.

Technically it could be done by the following steps:

- Choosing the git branch for the version that needs to be tested.
- Since Gemilo production is running in docker, that means the same docker files will be used but with different configurations.
- Building custom docker image for testing using official cypress image for copying cypress tests and configuring the docker file to copy the necessary requirement such as npm commands.
- Building images for the test database so we can start from a new database every time we run the test and configure the docker file for resetting the database and copying test dumbs before running the test.
- The previous steps will be automated in Jenkins CI environment.

8 References

Cypress (2022). Why cypress. cypress. docs.cypress.io/guides/overview/why-cypress#what-you-ll-learn

Docs. (n.d*).what is E2E Testing* https://docs.cypress.io/guides/core-concepts/testing-types#What-is-E2E-Testing

Docs.(n.d). What is Component Testing https://docs.cypress.io/guides/core-concepts/testing-types#What-is-Component-Testing

Docs. (2022).why is Testing Necessary? https://www.toolsqa.com/software-testing/istqb/why-is-testing-necessary/

Docs.(2022).Writing and organizing tests#Folder-structure https://docs.cypress.io/guides/core- concepts/writing-and-organizing-tests#Folder-structure

Docs. (2022). Writing and organizing tests#Hookshttps://docs.cypress.io/guides/core-concepts/writing-and-organizing-tests#Hooks

Docs. (n.d).Getting started https://testcafe.io/documentation/402635/getting-started

Da Costa, L. F. (2021). Testing JavaScript Applications. Simon and Schuster.

Everett, G. D., & Jr, R. M. (2007). Testing Across the Entire Software Development Life Cycle. Gerald D.Everett.

Garzone, M. A. (2014). Software Testing: A Guide to Testing Mobile Apps, Websites, and Games. CreateSpace.

Hamilton, T. (2021). White Box Testing – What is, Techniques, Example & Types. Guru99. https://www.guru99.com/white-box-testing.html

Hamilton, T. (2021). What is BLACK Box Testing? Techniques, Example & Types. Guru99. https://www.guru99.com/black-box-testing.html

Hamilton, T. (2022). Test plan: what is, how to create(with Example), Example & Types. Guru99. https://www.guru99.com/what-everybody-ought-to-know-test-planing.html

Kollewe, J & Wearden, G. (2015). Bloomberg IT meltdown leaves the financial world in the dark.

Javatpoint (n.d.), Levels of testing. https://www.javatpoint.com/levels-of-testing

Mwaura, W. (2021). End-to-End Web Testing with Cypress: Explore techniques for automated frontend web testing with Cypress and JavaScript. Packt Publishing Ltd. Axelrod, A. (2018). Complete Guide to Test Automation. Matan: Apress.

Nordeen, A. (2021). Learn Software Testing in 24 Hours. Scribd.

Peres, H. (2018). Automation Software Tests Using Selenium. Simplissimo Livros Ltda.

Stellman, A., & Greene, J. (2014). The Full Stack Developer. Andrew Stellman & Jennifer Greene.

Annex 2: Annex title

For the next annex, a section is added (Layout> Replacements> Section, Next Page). In the header of a new annex, edit two things: unlink the previous section (Header> Structure> Go> Link to Previous: No) and only then correct the annex number.